

Al-Aqsa University
Faculty of Computers and IT
Dept. of Info & Computer science



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم علم الحاسوب والمعلومات

Advanced Database Systems

Assignment **1**: Transaction Management and concurrency control

Presented to: **Eng. Mohammed Abohayya**

Student name: **Deema Mohammed AL-Maqadma**
Student number: **2320230766**

Semester: First Semester 2025/2026

1. What is a Transaction?

A transaction is a logical unit of work that must be entirely completed or aborted. It consists of one or more database requests, such as a SELECT statement, a series of related UPDATE statements, a series of INSERT statements, or a combination of SELECT, UPDATE, and INSERT statements.

Transactions ensure that the database remains in a consistent state, satisfying all integrity constraints.

For example:

```
SELECT CUSTNUMBER, CUSTBALANCE
FROM CUSTOMER
WHERE CUST_NUMBER = 10016;
```

```
INSERT INTO INVOICE VALUES (1009, 10016, '18-Jan-2018', 256.99, 20.56,
277.55, 'cred', 0.00, 277.55);
UPDATE PRODUCT SET PRODQOH = PRODQOH - 1 WHERE PROD_CODE =
'89-WRE-Q';
COMMIT;
```

2. ACID Properties of Transactions

Atomicity: All operations of a transaction must be completed. If not, the transaction is aborted.

Consistency: Ensures the permanence of the database's consistent state.

Isolation: Data used during a transaction cannot be accessed by another transaction until the first is completed.

Durability: Once transactions are committed, they cannot be undone or lost.

Serializability: Ensures that the schedule for concurrent execution of several transactions yields consistent results.

3. Transaction Log

The transaction log keeps track of all transactions that update the database.

It stores information such as:

- A record for the beginning of the transaction.
- The type of operation being performed (INSERT, UPDATE, DELETE).
- The names of the objects affected by the transaction.
- The "before" and "after" values for the fields being updated.
- Pointers to the previous and next transaction log entries for the same transaction.
- The ending (COMMIT) of the transaction.

The transaction log is critical for recovery because the DBMS uses it to roll back incomplete transactions or to restore the database after a system failure.

4. Concurrency Control

Concurrency control is the coordination of simultaneous transaction execution in a multiuser database system.

Its objective is to ensure serializability of transactions in a multiuser environment.

Serializability means that the interleaved execution of transactions produces the same result as if they were executed serially.

5. Problems in Concurrency Control

Lost Update: Occurs when two concurrent transactions update the same data element, and one of the updates is lost.

Example: T1 adds 100 units to PROD_QOH, while T2 subtracts 30 units at the same time. The addition may be lost.

Uncommitted Data: Occurs when a transaction reads data from another transaction that has not yet been committed.

Example: T1 updates PROD_QOH but then rolls back, while T2 has already read the uncommitted value.

Inconsistent Retrievals: Occurs when a transaction reads data before and after another transaction modifies it.

Example: T1 calculates the sum of PROD_QOH while T2 is updating some product quantities, leading to inconsistent totals.

6. Locking Methods

Locking guarantees exclusive use of a data item to a current transaction.

Levels of lock granularity include:

- Database-level lock
- Table-level lock
- Page-level lock
- Row-level lock
- Field-level lock

Shared lock: Allows concurrent transactions to read but not update a data item.

Exclusive lock: Reserved for the transaction that modifies the data item.

7. Two-Phase Locking (2PL)

Two-Phase Locking defines how transactions acquire and release locks.

- Growing phase: Transaction acquires all required locks without unlocking any data.
- Shrinking phase: Transaction releases all locks and cannot obtain new ones.

2PL guarantees serializability but does not prevent deadlocks.

8. Time Stamping Methods

Time stamping assigns a unique time stamp to each transaction.

The DBMS uses these time stamps to order transactions and ensure serializability.

Benefits: Avoids deadlocks.

Disadvantages: May cause transactions to be rolled back frequently if conflicts occur.

9. Deadlocks

A deadlock occurs when two or more transactions wait indefinitely for each other to release locks.

Techniques:

- Prevention: Avoid conditions that lead to deadlocks.
- Detection: Identify deadlocks when they occur.
- Avoidance: Use algorithms to ensure the system never enters a deadlock state.

10. Optimistic Concurrency Control

Optimistic concurrency control assumes conflicts are rare.

Phases:

- Read phase: Transaction reads data and performs computations.
- Validation phase: DBMS checks for conflicts.
- Write phase: If validation succeeds, changes are written to the database.

This method is preferred in environments with low conflict probability.

11. Database Recovery Management

Database recovery management ensures the database can be restored to a consistent state after a failure.

Techniques:

- Deferred-Write: Updates are not written to the database until the transaction reaches COMMIT.
- Write-Through: Updates are written immediately to the database.

Example:

Deferred-Write is useful when many transactions may be rolled back.

Write-Through is useful when durability of each update must be guaranteed immediately.

" واخر دعواهم ان الحمد لله رب العالمين "